Claude Code: Asking Orthogonal Questions Without Disturbing Context

Conversation transcript — July 17, 2026

QUESTION 1

Suppose that I am in a claude code conversation about a code project and have built a significant context. Is there a way to ask an orthogonal question without disturbing the project context

RESPONSE

Yes — Claude Code has a **/btw** command built exactly for this. **How it works:**

• You type /btw <your question> while in your main session.
• Claude answers in a temporary overlay that appears on screen.
• Press Space, Enter, or Esc to dismiss it.
• The question and answer are **not** added to your conversation history — your context stays exactly as it was.

It even works while Claude Code is mid-task (editing files, running tests, etc.) — the side question runs independently and doesn't interrupt the ongoing work.

Important limitation: /btw has no tool access — it can't read files, run commands, or search your codebase. It can only answer from what's already in the current conversation (files it's read, decisions made, things discussed). So it's great for "what did we decide about caching?" or "remind me what that function signature was" but not for "go check what's in this new file."

If you need something Claude hasn't seen yet: that's what subagents are for — they get full tool access but start with a blank context, the opposite tradeoff of /btw. One rule of thumb from Anthropic's docs: for a quick question about something already in your conversation, use /btw instead of a subagent, since it sees your full context but has no tool access, and the answer is discarded rather than added to history.

So for your case — an orthogonal question during a long project session — /btw is almost certainly the right tool, as long as the question doesn't require Claude to go read new files or run anything.

QUESTION 2

Are subagent's context independent of the current conversation context?

RESPONSE

Yes — subagent context is fully independent (isolated) from the main conversation.

Key points from Claude Code's docs:

• **Fresh start:** Each subagent invocation creates a new instance with fresh context. It doesn't inherit the history, files read, or decisions made in your main session — it starts essentially blank.

• **Purpose is isolation:** Subagents are separate agent instances that your main agent can spawn to handle focused subtasks, used to isolate context for focused subtasks, run multiple analyses in parallel, and apply specialized instructions without bloating the main agent's prompt.

• **One-way handoff:** The subagent does its work (with full tool access — reading files, running commands, etc.), then returns a summary/result to the main conversation. Only that returned result gets added to your main context — not the subagent's internal exploration, tool calls, or verbose output. That's the token-saving benefit: a subagent might read 10 files and burn 20k tokens doing so, but your main thread only absorbs the final summary.

• **No context sharing back either:** if you need a subagent to continue reasoning with context from an earlier subagent run, you have to explicitly resume that specific subagent session (per the SDK docs, this requires capturing and passing the session ID) — it's not automatic.

So the trade-off versus /btw is exactly inverted:

/btw: Sees main conversation context (yes, full visibility). Has tool access (no). Adds to main context (no — discarded after you dismiss it).

Subagent: Sees main conversation context (no, starts fresh). Has tool access (yes). Adds to main context (only the final returned summary is added).

So if your orthogonal question requires exploring code or running something the subagent hasn't already seen, a subagent is the right call — but remember it won't know anything about your current project state unless you explicitly tell it in the prompt you give it.